

Here the content would be directly stored in the ROM and the program will have to look up the ROM in case it wants to access data from this variable.

Volatile

When a programmer declares a variable to be volatile, it signals both the compiler and the developer that the value of this variable can change anytime and this can happen by means which are outside the normal flow of the application. The changes can be caused by a peripheral, another processor in a multiprocessor system or an interrupt. The signal to the compiler in this case is to ask it to not optimise this variable. Usually volatile is used for input and output devices which are stored as variables in C.

Now, let us see (pun not intended!) how we can write the blinking LED program for an ARCOM board. The blinking LED program is the “Hello World!” program of embedded system. It is usually used by the programmers to get a feel of how to program for a given board – it’s features and specialities which they can further use for the task at hand.

```
#include "led.h"

int main(void) {
    /* Configure the green LED control pin. */
    ledInit();

    while (1) {
        /* Toggle the state of the green LED. */
        ledToggle();

        /* Pause for 500 ms */
        delay_ms(500);
    }
    return 0;
}
```

This is the main function which controls the program’s flow. We next define the helper functions – ledInit, ledToggle() and delay_ms().

Given that there is a PXA255 Processor on the board, we next have to go through the developer documentation to find out which pins control the onboard LED. According to the PXA255 Processor Developer’s the